Economics Architecture for AI Coding Agent Harnesses

First Author[1], **Second Author**[2], **Third Author**[2], **Fourth Author**[1] **and Fifth Author**[1]
[1]Pragnition Labs, [*]Equal Contribution, [1]Affiliation 1, [2]Affiliation 2

AI coding agents consume millions of tokens across multi-stage pipelines, yet no existing harness includes an explicit cost optimization layer. We present a formal framework grounding economics architecture in five interlocking results. First, we formulate the Token Budget Allocation Problem (TBAP), where quality is multiplicative across pipeline stages while cost is additive; a log-transformation converts this to a concave program admitting a water-filling solution and an Equal Marginal Rate theorem. Second, we formalize model tier selection as a variant of the Generalized Assignment Problem (GAP), proving NP-hardness in general but establishing tractability via enumeration for practical instance sizes (3–5 tiers, 5–8 stages). Third, we analyze the dual-regime cost function (interactive versus autonomous) through queueing economics, deriving the M/M/1 optimal service rate that determines model selection per regime. Fourth, we formalize the Jevons paradox for token markets, proving that cost optimization becomes *more* necessary as per-token prices fall when demand elasticity exceeds unity (empirically estimated at $\epsilon \approx 1.19$). Fifth, we introduce the CVIH metric (cost-adjusted verified iterations per hour) as the true efficiency measure, proving it is quasi-concave with a unique optimal budget strictly below the quality optimum. We derive seven design principles, prove a Weakest-Link theorem and a Cache-First principle, and calibrate all models against April 2026 pricing data reflecting a 300× price reduction over three years, 90–95% cache hit rates, and 60× model tier price spreads.

1. Introduction

Cost is the binding constraint for AI coding agent deployment at scale. While research attention has focused on capability benchmarks (SWE-bench pass rates, code generation accuracy), practitioners face a more immediate challenge: a single 50-iteration agentic coding session can consume 2–5 million tokens, costing $3–$50 depending on model tier and caching configuration. Organizations running hundreds of such sessions daily confront annual token budgets in the range of $200K–$2M, placing AI coding costs in the same category as cloud infrastructure expenditure (FinOps Foundation, 2023).

The R3 reviewer of a companion paper captured this tension precisely: "The paper does not sufficiently address the economics of these harnesses. Token costs, model selection, and budget allocation deserve formal treatment, not just a pricing table." This paper answers that call.

Quantitative motivation. Consider a typical agentic coding pipeline with $K = 6$ stages (planning, context retrieval, code generation, testing, review, iteration). At April 2026 pricing, a single iteration through this pipeline consumes roughly 40K–80K tokens. A task requiring 30–50 iterations thus demands 1.2M–4M tokens. With parallel execution across 5–10 developer workstreams, daily organizational consumption reaches 50M–200M tokens. At frontier model prices ($3–$15 per million input tokens), this translates to $150–$3,000 per day before caching discounts. The economics are not negligible; they are a first-order architectural concern.

Table 1 | Cost optimization features in existing AI coding agent harnesses. A checkmark (✓) indicates the feature is present; a cross (✗) indicates absence.

Feature	GSD	RAPID	Turing	Blueprint	Cursor	Codex
Explicit budget allocation	✗	✗	✗	✗	✗	✗
Per-stage model routing	✗	✗	✗	✗	Partial	✗
Cache-aware optimization	✗	✗	✗	✗	Provider	Provider
Cost-quality Pareto analysis	✗	✗	✗	✗	✗	✗
Regime detection (interactive/autonomous)	✗	✗	✗	✗	✗	✗
Jevons-aware demand forecasting	✗	✗	✗	✗	✗	✗

The missing layer. Table 1 surveys existing harness architectures. None includes an explicit cost optimization layer. GSD and RAPID treat model selection as a configuration parameter. Turing delegates cost management to the user. Cursor and Codex rely on provider-side optimizations (caching, rate limits) without harness-level cost awareness. Blueprint tracks architectural decisions but does not model their economic consequences.

Contributions. This paper makes the following contributions:

1. **The Token Budget Allocation Problem (TBAP)** with multiplicative quality and additive cost, solved via log-transformation yielding a water-filling solution (section 3).
2. **The Model Tier Selection Problem (MTSP)** formalized as a GAP variant, with NP-hardness in general and tractability by enumeration for practical instance sizes (section 4).
3. **Dual-regime queueing economics** (interactive versus autonomous) with the M/M/1 optimal service rate determining model selection (section 5).
4. **Formal Jevons paradox for token markets**, proving that cost optimization becomes more necessary as prices fall (section 6).
5. **The CVIH metric** and its optimization properties, including quasi-concavity and the result that the CVIH-optimal budget lies strictly below the quality optimum (section 7).
6. **Seven design principles** derived from the formal results, each grounded in a theorem or empirical calibration (section 9).
7. **Empirical calibration** from April 2026 pricing data across Anthropic, OpenAI, and Google, with caching parameters and developer time estimates (section 10).

Scope and novelty. We make no claim to inventing portfolio theory, queueing theory, or the Jevons paradox. Our contribution lies in the application and synthesis: showing that the specific structure of AI coding agent pipelines (multiplicative quality, additive cost, dual operating regimes, extreme price dynamics) creates a distinctive optimization landscape that existing frameworks do not directly address. The formal results in sections 3 and 7 are, to our knowledge, novel in this application domain. The empirical models in section 10 are precisely that: empirical, calibrated against current data, and subject to revision as the market evolves.

2. Preliminaries and Definitions

2.1. Pipeline Model

We model an AI coding agent harness as a pipeline of K stages, indexed $k \in \{1, \ldots, K\}$. Each stage k receives a token allocation $r_k \geq 0$, uses a model with per-token price $p_k > 0$, and produces output

of quality $q_k(r_k) \in [0, 1]$.

Definition 2.1 (Pipeline)**.** *A* pipeline *is a tuple* $\Pi = (K, \mathrm{r}, \mathrm{p}, \mathrm{q})$ *where:*

- $K \in \mathbb{N}$ *is the number of stages,*
- $\mathrm{r} = (r_1, \ldots, r_K) \in \mathbb{R}^K_{\geq 0}$ *is the token allocation vector,*
- $\mathrm{p} = (p_1, \ldots, p_K) \in \mathbb{R}^K_{>0}$ *is the price vector,*
- $\mathrm{q} = (q_1, \ldots, q_K)$ *is the quality function vector.*

In practice, K ranges from 4 to 8 for typical harnesses, and each r_k represents thousands to hundreds of thousands of tokens.

2.2. Quality Function Axioms

We require quality functions to satisfy the following axioms, which codify empirically observed behavior of LLM performance as a function of token budget.

Definition 2.2 (Quality function axioms)**.** *A function* $q : \mathbb{R}_{\geq 0} \to [0, 1]$ *is a* quality function *if it satisfies:*

(Q1) Boundary: $q(0) = 0$.
(Q2) Monotonicity: q *is non-decreasing.*
(Q3) Boundedness: $\lim_{r\to\infty} q(r) \leq 1$.
(Q4) Concavity: q *is concave on* $\mathbb{R}_{\geq 0}$.
(Q5) Differentiability: q *is continuously differentiable on* $\mathbb{R}_{>0}$.

Remark. *Axiom* ***(Q4)****, concavity, is the critical structural assumption. It encodes diminishing returns: the first thousand tokens of context contribute more to quality than the hundred-thousandth. Empirical evidence from Gao et al. (2025) and Liu et al. (2024) supports this for retrieval-augmented and long-context settings. However, concavity is an idealization; real quality functions may exhibit non-monotone behavior at extreme context lengths ("context rot", Chroma Research, 2025). We address this in section 9 with the "Cap the Context" principle.*

2.3. Multiplicative Quality, Additive Cost

The key structural feature of our model is the asymmetry between quality aggregation and cost aggregation.

Definition 2.3 (Pipeline quality and cost)**.** *Given a pipeline* Π*, the* aggregate quality *and* aggregate cost *are:*

$$Q(\mathrm{r}) = \prod_{k=1}^{K} q_k(r_k), \tag{1}$$

$$C(\mathrm{r}) = \sum_{k=1}^{K} p_k \cdot r_k. \tag{2}$$

The multiplicative structure of eq. (1) reflects a fundamental operational reality: if any single stage fails completely ($q_k = 0$), the entire pipeline output is worthless. A brilliant code generation (stage 3) is useless if the planning stage (stage 1) misidentified the task, or if the test stage (stage 4) fails to validate correctness. This is the software reliability series-system model of Barlow and Proschan (1975), applied to cognitive pipeline stages.

The additive structure of eq. (2) is straightforward: token costs accumulate linearly. There are no cross-stage cost interactions (aside from caching, which we model separately in section 8).

This asymmetry, multiplicative quality but additive cost, is what makes the optimization problem interesting and is the source of most of our formal results.

2.4. The CVIH Metric

We now define the efficiency metric that will serve as our primary optimization objective.

Definition 2.4 (CVIH)**.** *The* cost-adjusted verified iterations per hour *(CVIH) is:*

$$CVIH = \frac{\lambda_{raw} \cdot p_{verify}}{C_{total}}, \tag{3}$$

where λ_{raw} is the raw iteration rate (iterations per hour), $p_{verify} \in [0, 1]$ is the probability that an iteration produces a verified-correct result, and C_{total} is the total cost per iteration.

CVIH measures *verified progress per dollar-hour*. It is the natural efficiency metric for cost-constrained agentic systems: maximizing CVIH means getting the most verified work done per unit of combined time and money.

Remark. *CVIH is not directly computable from first principles; it depends on empirical estimates of λ_{raw} and p_{verify}, both of which are functions of the token allocation and model selection. We treat CVIH as an empirical metric that can be* optimized *using the formal framework, not as a quantity that can be computed from axioms alone. This honest framing distinguishes CVIH from metrics that claim formal status while depending on unobservable quantities.*

3. The Token Budget Allocation Problem

3.1. Problem Formulation

Definition 3.1 (TBAP)**.** *The* Token Budget Allocation Problem *is:*

$$\begin{aligned} \underset{\mathrm{r} \in \mathbb{R}^K_{\geq 0}}{\text{maximize}} \quad & Q(\mathrm{r}) = \prod_{k=1}^{K} q_k(r_k) \\ \textit{subject to} \quad & \sum_{k=1}^{K} p_k \cdot r_k \leq B, \end{aligned} \tag{4}$$

where $B > 0$ is the total budget.

The TBAP asks: given a fixed token budget, how should tokens be allocated across pipeline stages to maximize the probability that the entire pipeline succeeds?

3.2. Log-Transformation

The multiplicative objective is inconvenient for direct optimization. A standard technique converts it to an additive form.

Proposition 3.1 (Log-transformation equivalence)**.** *The TBAP (eq. (4)) is equivalent to:*

$$\begin{aligned} \underset{\mathrm{r}\in\mathbb{R}^K_{\geq 0}}{\text{maximize}} \quad & \sum_{k=1}^{K} \log q_k(r_k) \\ \textit{subject to} \quad & \sum_{k=1}^{K} p_k \cdot r_k \leq B, \\ & r_k > 0 \quad \forall k, \end{aligned} \tag{5}$$

in the sense that both problems have the same set of optimal solutions (restricting to $\mathrm{r} > 0$*).*

Proof. Since $\log$ is strictly increasing on $(0, \infty)$, maximizing $\prod_k q_k(r_k)$ over the feasible set where all $q_k(r_k) > 0$ is equivalent to maximizing $\sum_k \log q_k(r_k)$. By axiom **(Q1)**, $q_k(0) = 0$, so the optimal solution must have $r_k > 0$ for all k (otherwise $Q = 0$). □

Proposition 3.2 (Concavity of transformed objective)**.** *Under axioms **(Q1)–(Q5)**, the function* $f(\mathrm{r}) = \sum_{k=1}^{K} \log q_k(r_k)$ *is concave on* $\{r \in \mathbb{R}^K_{>0}\}$.

Proof. Each q_k is concave and maps into $(0, 1] \subset (0, \infty)$ (axioms **Q1–Q4**). The function $\log$ is concave and non-decreasing on all of $(0, \infty)$. By the composition rule for concave functions (Boyd and Vandenberghe, 2004, Section 3.2.4), if h is concave and non-decreasing and g is concave, then $h \circ g$ is concave. Applying this with $h = \log$ and $g = q_k$ yields that $\log \circ q_k$ is concave on the domain where $q_k > 0$. Since f is a sum of concave functions, it is concave. □

3.3. The Equal Marginal Rate Theorem

With concavity established, the TBAP is a concave maximization problem with a linear constraint. The Karush-Kuhn-Tucker (KKT) conditions characterize its solution.

Theorem 3.1 (Equal Marginal Rate)**.** *Let* r^* *be an optimal solution to the TBAP with* $r_k^* > 0$ *for all* k*. Then there exists* $\lambda^* \geq 0$ *such that for every stage* k*:*

$$\frac{q_k'(r_k^*)}{q_k(r_k^*) \cdot p_k} = \lambda^*. \tag{6}$$

That is, the marginal quality return per dollar *is equalized across all stages at optimality.*

Proof. The KKT conditions for the log-transformed problem (eq. (5)) require:

$$\frac{\partial}{\partial r_k} \sum_{j=1}^{K} \log q_j(r_j) = \lambda^* \cdot p_k \quad \forall k,$$

which gives $q_k'(r_k^*)/q_k(r_k^*) = \lambda^* p_k$. Dividing both sides by p_k yields eq. (6). Existence of λ^* follows from the constraint qualification (Slater's condition holds since the feasible set has non-empty interior). See section A for the complete argument. □

Theorem 3.2 (Uniqueness)**.** *If each* q_k *is strictly concave on* $\mathbb{R}_{>0}$*, the optimal allocation* r^* *is unique.*

Proof. Strict concavity of each q_k implies strict concavity of $\log q_k$ (since $\log$ is strictly concave and q_k is strictly concave and positive). Therefore $f(\mathrm{r}) = \sum_k \log q_k(r_k)$ is strictly concave, and a strictly concave function on a convex set has at most one maximizer (Rockafellar, 1970). □

Interpretation. Theorem 3.1 is the token-economics analogue of the equal marginal utility per dollar condition from consumer theory (Mas-Colell et al., 1995). The ratio $q_k'(r_k^*)/(q_k(r_k^*) \cdot p_k)$ measures the percentage improvement in stage-k quality per additional dollar spent. At optimality, this ratio is the same for all stages. If it were higher for some stage j, one could improve total quality by shifting budget from a low-ratio stage to stage j.

3.4. The Weakest-Link Theorem

The multiplicative structure implies a strong equalization result that goes beyond equal marginal rates.

Theorem 3.3 (Weakest-Link, Symmetric Case)**.** *In the special case where all stages have identical quality functions ($q_k = q$ for all k) and identical prices ($p_k = p$ for all k), the optimal allocation is uniform: $r_k^* = B/(Kp)$ for all k.*

Proof. By the EMR condition (theorem 3.1), at optimality $q'(r_k^*)/(q(r_k^*) \cdot p) = \lambda^*$ for all k. Since q is strictly concave and p is identical across stages, this requires $r_k^* = r_j^*$ for all k, j. The budget constraint then gives $r_k^* = B/(Kp)$. □

The symmetric case illustrates a more general principle. For *any* allocation (not necessarily optimal), the AM-GM inequality bounds the product:

Proposition 3.3 (AM-GM Quality Bound)**.** *For any allocation* r *with quality values $q_k(r_k)$:*

$$\prod_{k=1}^{K} q_k(r_k) \leq \left(\frac{1}{K} \sum_{k=1}^{K} q_k(r_k) \right)^K, \tag{7}$$

with equality if and only if $q_1(r_1) = q_2(r_2) = \cdots = q_K(r_K)$.

This bound is a property of quality *values*, not directly of budget allocations. It establishes that for a fixed sum of stage qualities, the product is maximized when qualities are equalized. The connection to budget allocation is mediated by the quality functions: in the symmetric case, equal budgets yield equal qualities; in the heterogeneous case, the EMR condition drives the optimizer to approximately equalize quality levels, investing more in stages with lower baseline quality. See section A for the full proof.

Corollary 3.1 (Raise the Floor)**.** *Given a current allocation* r *with $q_j(r_j) < q_k(r_k)$ for some stages j, k, the marginal return from increasing r_j (the weaker stage) exceeds the marginal return from increasing r_k (the stronger stage), provided both are in the concave region.*

This is the formal basis for the "raise the floor" design principle: invest disproportionately in the weakest pipeline stage. A 10% improvement in the worst stage yields a larger multiplicative quality gain than a 10% improvement in the best stage.

3.5. The Water-Filling Analogy

The structure of the TBAP solution is identical to the classical water-filling solution for parallel Gaussian channels (Cover and Thomas, 2006). In that setting, one allocates power across channels to maximize total capacity; the optimal allocation "fills" channels with the lowest noise first, up to a common water level.

In our setting, the "channels" are pipeline stages, "power" is token budget, and "noise" is the inverse of quality function responsiveness. The common "water level" is the Lagrange multiplier λ^* from theorem 3.1. Stages with high price p_k or low responsiveness $q_k'(\cdot)$ receive less budget, analogous to noisy channels receiving less power.

3.6. Pareto Frontier Characterization

As the budget B varies, the optimal quality $Q^*(B)$ traces out the cost-quality Pareto frontier.

Proposition 3.4 (Pareto frontier properties)**.** *Under axioms **(Q1)–(Q5)**, the function $Q^* : \mathbb{R}_{>0} \to (0, 1]$ defined by $Q^*(B) = \max_{\mathrm{r}}\{Q(\mathrm{r}) : C(\mathrm{r}) \le B\}$ satisfies:*

1. *Q^* is non-decreasing in B.*
2. *$\log Q^*$ is concave in B (i.e., Q^* is log-concave).*
3. *The slope of the Pareto frontier at budget B equals the optimal Lagrange multiplier: $\frac{d}{dB} \log Q^*(B) = \lambda^*(B)$.*

Proof. Property (1) follows from feasibility: increasing B enlarges the feasible set. Property (2) follows from the concavity of the log-transformed objective in r and the convexity of the feasible set in B (see section A). Property (3) is the envelope theorem applied to the parametric optimization problem. □

4. Model Tier Selection and Joint Optimization

4.1. The Model Tier Selection Problem

In practice, the price p_k is not fixed for each stage; it depends on which model is assigned to that stage. Let $\mathcal{M} = \{m_1, \ldots, m_M\}$ be a set of available models, each with price $p^{(i)}$ and a stage-specific capability modifier $\gamma_k^{(i)} \in [0, 1]$ that scales the quality function: if model i is assigned to stage k, then $q_k^{(i)}(r) = \gamma_k^{(i)} \cdot q_k(r)$.

Definition 4.1 (MTSP)**.** *The* Model Tier Selection Problem *is:*

$$\begin{aligned} \underset{\sigma:[K]\to[M]}{\text{minimize}} \quad & \sum_{k=1}^{K} p^{(\sigma(k))} \cdot r_k \\ \textit{subject to} \quad & \gamma_k^{(\sigma(k))} \ge \gamma_k^{\min} \quad \forall k, \\ & \prod_{k=1}^{K} \gamma_k^{(\sigma(k))} \cdot q_k(r_k) \ge Q_{\min}, \end{aligned} \tag{8}$$

where σ is the model assignment function, $\gamma_k^{\min}$ is the minimum capability required at stage k, and $Q_{\min}$ is the minimum acceptable pipeline quality.

4.2. Relationship to GAP

The MTSP is a variant of the Generalized Assignment Problem (GAP), in which items (stages) must be assigned to bins (models) subject to capacity and quality constraints (Chekuri and Khanna, 2005). The GAP is NP-hard in general, but admits polynomial-time approximation schemes for fixed numbers of bins.

Proposition 4.1 (NP-hardness)**.** *The MTSP is NP-hard in general (when M and K are both part of the input).*

Proof sketch. We reduce from the 0-1 knapsack problem. Given a knapsack instance with n items (weights w_i, values v_i) and capacity W, construct an MTSP instance with $K = n$ stages and $M = 2$ models. Model 1 has cost $p^{(1)} = 0$ and quality modifier $\gamma_k^{(1)} = 1$ for all stages; model 2 has cost $p^{(2)} = 1$ and quality modifier $\gamma_k^{(2)} = e^{v_k}$. Set token demands $r_k = w_k$, budget $B = W$, and quality floor $Q_{\min} = e^V$ where V is the required total value. An assignment σ maps each stage to model 1 ("exclude item") or model 2 ("include item"). The cost $\sum_{k:\sigma(k)=2} w_k \le W$ encodes the weight constraint; the quality $\prod_k \gamma_k^{(\sigma(k))} = e^{\sum_{k:\sigma(k)=2} v_k} \ge e^V$ encodes the value constraint. Since 0-1 knapsack is NP-hard, so is MTSP. □

Proposition 4.2 (Practical tractability)**.** *For fixed M (number of model tiers), the MTSP is solvable in $O(M^K)$ time by enumeration. For typical values ($M \in \{3,4,5\}$ and $K \in \{5,6,7,8\}$), this yields at most $5^8 = 390{,}625$ candidate assignments, which is tractable.*

Proof. The assignment function $\sigma : [K] \to [M]$ has M^K possible values. Each can be checked against the constraints in $O(K)$ time. For the stated parameter ranges, $M^K \le 5^8 = 390{,}625$. □

4.3. Joint Allocation-Selection Problem

The full optimization problem combines TBAP and MTSP.

Definition 4.2 (JASP)**.** *The* Joint Allocation-Selection Problem *is:*

$$\begin{aligned} \underset{\sigma,\mathrm{r}}{\text{maximize}} \quad & \prod_{k=1}^{K} \gamma_k^{(\sigma(k))} \cdot q_k(r_k) \\ \textit{subject to} \quad & \sum_{k=1}^{K} p^{(\sigma(k))} \cdot r_k \le B, \\ & \gamma_k^{(\sigma(k))} \ge \gamma_k^{\min} \quad \forall k. \end{aligned} \tag{9}$$

The JASP decomposes naturally: for each candidate assignment σ, the inner problem (optimizing r given σ) is a TBAP instance, which is convex and efficiently solvable. The outer problem enumerates over at most M^K assignments.

Worked example. Consider 3 model tiers and 6 stages. The frontier model (Tier 1) costs \$15/Mtok with $\gamma = 1.0$; the mid-tier (Tier 2) costs \$3/Mtok with $\gamma \in [0.7, 0.95]$ depending on stage; the economy tier (Tier 3) costs \$0.25/Mtok with $\gamma \in [0.4, 0.8]$. With $3^6 = 729$ candidate assignments, each requiring a convex optimization over 6 variables, the full JASP solves in under one second on commodity hardware. The optimal assignment typically places the frontier model on the most quality-sensitive stages (planning, code generation) while routing less sensitive stages (formatting, simple retrieval) to economy tiers, achieving 70–85% of the all-frontier quality at 20–35% of the cost.

5. Queueing Economics of Developer Waiting Time

5.1. The Dual-Regime Cost Function

AI coding agents operate in two fundamentally different regimes:

- **Interactive regime:** The developer waits for each response, maintaining a conversational loop. Latency directly impacts flow state and productivity.
- **Autonomous regime:** The agent runs independently (background execution, batch processing). Latency matters only at the granularity of task completion, not individual responses.

These regimes have different cost structures. In the interactive regime, developer waiting time has high opportunity cost; in the autonomous regime, it has near-zero opportunity cost (the developer works on other tasks). This duality has profound implications for model selection.

Definition 5.1 (Effective cost)**.** *The* effective cost per token *in regime $\mathcal{R}$ is:*

$$p_{eff}^{(\mathcal{R})} = p_m + \frac{c_w^{(\mathcal{R})}}{s_m}, \tag{10}$$

where p_m is the per-token price of model m, $c_w^{(\mathcal{R})}$ is the cost of developer waiting time per unit time in regime $\mathcal{R}$, and s_m is the throughput of model m (tokens per unit time).

In the interactive regime, $c_w^{(\text{int})}$ equals the developer's fully loaded hourly rate ($90–$200/hr). In the autonomous regime, $c_w^{(\text{auto})} \approx 0$ since the developer is productive elsewhere.

5.2. M/M/1 Optimal Service Rate

We model the interactive regime as an M/M/1 queue with arrival rate λ (developer requests per unit time) and service rate μ (model response rate), following Kleinrock (1975).

Proposition 5.1 (Optimal service rate)**.** *In the M/M/1 model with waiting cost c_w per unit time and service cost c_s per unit service rate, the optimal service rate is:*

$$\mu^* = \lambda + \sqrt{\frac{c_w \cdot \lambda}{c_s}}. \tag{11}$$

Proof. The expected waiting time in an M/M/1 queue is $W = 1/(\mu - \lambda)$ (Kleinrock, 1975). The total cost rate is $c_w \cdot \lambda \cdot W + c_s \cdot \mu = c_w \lambda/(\mu - \lambda) + c_s \mu$. Taking the derivative with respect to μ, setting it to zero, and solving yields eq. (11). The second-order condition confirms this is a minimum. □

Interpretation. The optimal service rate exceeds the arrival rate by $\sqrt{c_w \lambda / c_s}$. When developer time is expensive (c_w large) or request volume is high (λ large), one should invest in faster (more expensive) models. When service cost is high (c_s large), one should tolerate more waiting.

5.3. The $c\mu$ Priority Rule

For mixed workloads containing both interactive and autonomous tasks, the classical $c\mu$ rule (Van Mieghem, 1995) dictates priority ordering:

Proposition 5.2 ($c\mu$ rule for mixed workloads)**.** *In a single-server system with multiple task classes, the policy that minimizes total weighted waiting cost processes tasks in decreasing order of $c_j \mu_j$, where c_j is the per-unit-time waiting cost of class j and μ_j is the service rate for class j.*

For AI coding agents, this implies interactive tasks (high c_j) should always be prioritized over autonomous tasks (low c_j), and within each regime, tasks with faster expected completion should be prioritized.

5.4. The Flow-State Step Function

Empirical research on interruption costs suggests that developer context-switching cost is not a smooth function of waiting time but rather a step function with a critical threshold.

Mark et al. (2008) found that interrupted knowledge workers take approximately 23 minutes to return to their original task context. Leroy (2009) identified "attention residue," where performance on the new task suffers because attention remains partially focused on the prior task.

For AI coding agents, we model the flow-state cost as:

$$c_{\text{flow}}(w) = \begin{cases} 0 & \text{if } w < w_{\text{thresh}}, \\ c_{\text{switch}} & \text{if } w \geq w_{\text{thresh}}, \end{cases} \tag{12}$$

where w is the waiting time per interaction, $w_{\text{thresh}} \approx 120$–$180$ seconds (based on empirical observation), and c_{switch} is the context-switching cost (estimated at 15–30 minutes of reduced productivity, or $25–$100 at loaded developer rates).

This step-function structure implies a clear model selection criterion for the interactive regime: choose the cheapest model whose response time is below w_{thresh}.

5.5. Regime-Optimal Model Selection

Combining the effective cost (definition 5.1) with the flow-state step function (eq. (12)):

- **Interactive regime:** Select the cheapest model m such that expected response time $\mathbb{E}[T_m] < w_{\text{thresh}}$. The effective cost is dominated by c_w/s_m.
- **Autonomous regime:** Select the model m that minimizes p_m/γ_m (cost per unit capability). The waiting cost term vanishes.

In practice, this often means using a faster, mid-tier model for interactive work and a slower, frontier model for autonomous batch processing, the opposite of the naive "use the best model for everything" strategy.

6. The Jevons Paradox for Token Economics

6.1. Formal Setup

The Jevons paradox (Jevons, 1865) states that improvements in efficiency of resource use tend to *increase* rather than decrease total resource consumption, because lower per-unit costs stimulate demand expansion. We formalize this for token markets.

Let p denote the per-token price, $D(p)$ the aggregate demand (tokens consumed), and $S(p) = p \cdot D(p)$ the total spend.

Definition 6.1 (Price elasticity of demand)**.** *The* price elasticity of demand *is:*

$$\epsilon = -\frac{p}{D(p)} \cdot \frac{dD}{dp}. \tag{13}$$

Theorem 6.1 (Jevons condition)**.** *Total spend $S(p) = p \cdot D(p)$ increases when price p decreases if and only if $\epsilon > 1$. That is:*

$$\frac{dS}{dp} < 0 \iff \epsilon > 1. \tag{14}$$

Proof.

$$\frac{dS}{dp} = D(p) + p \cdot D'(p) = D(p)\left(1 + \frac{p \cdot D'(p)}{D(p)}\right) = D(p)(1 - \epsilon).$$

Since $D(p) > 0$, we have $dS/dp < 0$ if and only if $1 - \epsilon < 0$, i.e., $\epsilon > 1$. □

6.2. Elasticity Estimation

Estimating ϵ for token markets requires aggregate data on price and consumption changes. From publicly available data (April 2023 to April 2026):

- Frontier model input prices fell from approximately $30/Mtok to approximately $3/Mtok (a roughly 10× reduction; with caching, the effective reduction is closer to 50×).
- Aggregate token consumption grew by an estimated 100–500× (based on provider capacity reports).

Using the midpoint formula for arc elasticity over this period yields $\epsilon \approx 1.1$–1.3, with a point estimate of $\epsilon \approx 1.19$. This places the token market firmly in the elastic regime ($\epsilon > 1$), confirming the Jevons condition.

Remark (Sensitivity and limitations of the elasticity estimate)**.** *This estimate has several caveats. First, the consumption growth range (100–500×) spans half an order of magnitude; at the lower bound, $\epsilon \approx 1.1$, while at the upper bound, $\epsilon \approx 1.3$. Second, the estimate conflates the intensive margin (existing users consuming more tokens per task) and the extensive margin (new users and use cases entering the market). If the extensive margin dominates (as we argue in section 6.3), the Jevons paradox applies to the* market *but not necessarily to individual organizations that do not adopt new use cases. An organization with fixed workflows may have $\epsilon < 1$ even while the aggregate market has $\epsilon > 1$. Third, no confidence interval can be constructed from two aggregate data points. Despite these limitations, the qualitative conclusion ($\epsilon > 1$ for the aggregate market) is robust: the direction is unambiguous (prices fell dramatically, spending rose), and even the conservative bound ($\epsilon \approx 1.1$) exceeds unity.*

6.3. Demand Decomposition

Token demand expansion has two components:

1. **Intensive margin:** Existing agentic pipelines use more tokens per task (longer contexts, more iterations, higher-quality prompts). This is the direct analogue of Jevons's coal efficiency scenario.
2. **Extensive margin:** Falling prices make previously uneconomical use cases viable (automated code review, speculative pre-computation, parallel hypothesis testing).

Both margins contribute to $\epsilon > 1$. The extensive margin is likely dominant in the current growth phase, as entirely new agentic workflows become cost-feasible each time prices fall by 2–3×.

6.4. Implication for Cost Optimization

Corollary 6.1 (Jevons imperative)**.** *When $\epsilon > 1$, falling per-token prices* increase *total organizational token spend. Therefore, the value of cost optimization (measured as potential savings) grows as prices fall.*

This is counterintuitive. The naive expectation is that falling prices eliminate the need for cost optimization. The Jevons paradox reverses this: falling prices stimulate demand expansion that more

than offsets the per-unit savings, making budget discipline and allocation optimization more valuable, not less. The bandwidth market of the 2000s provides a close historical analogy: as bandwidth prices collapsed, total telecom spending increased, and traffic engineering became more, not less, important (Sorrell, 2009).

7. The CVIH Metric and Budget Optimality

7.1. CVIH as an Optimization Objective

Recall from definition 2.4 that CVIH = $\lambda_{\text{raw}} \cdot p_{\text{verify}}/C_{\text{total}}$. We now study how CVIH depends on the total budget B allocated to the pipeline.

Both λ_{raw} and p_{verify} are functions of B: increasing the budget generally increases quality (p_{verify} rises) but may decrease throughput (λ_{raw} falls due to longer processing) and always increases cost ($C_{\text{total}} = B$ when the budget is fully spent). Let us define:

$$\text{CVIH}(B) = \frac{V(B)}{B}, \tag{15}$$

where $V(B) = \lambda_{\text{raw}}(B) \cdot p_{\text{verify}}(B)$ is the "value function" (verified iterations per hour as a function of budget).

Theorem 7.1 (CVIH tangent optimality)**.** *The budget B^* that maximizes $\text{CVIH}(B) = V(B)/B$ satisfies:*

$$V'(B^*) = \frac{V(B^*)}{B^*}, \tag{16}$$

i.e., the marginal value equals the average value. Geometrically, this is the point where the line from the origin is tangent to the value curve $V(B)$.

Proof. Setting $\frac{d}{dB}[V(B)/B] = 0$ gives $[V'(B) \cdot B - V(B)]/B^2 = 0$, hence $V'(B^*) = V(B^*)/B^*$. □

Theorem 7.2 (Quasi-concavity of CVIH)**.** *If $V(B)$ is concave, continuously differentiable, $V(0) = 0$, and $V'(0) > 0$, then $\text{CVIH}(B) = V(B)/B$ is quasi-concave on $(0, \infty)$ with a unique maximizer B^*.*

Proof. Since V is concave with $V(0) = 0$ and $V'(0) > 0$, the function $V(B)/B$ starts at $V'(0) > 0$ (by L'Hôpital's rule), is continuous, and eventually decreases (since V is bounded while $B \to \infty$). The ratio of a concave function to a linear function through the origin is quasi-concave (Boyd and Vandenberghe, 2004). The unique maximizer follows from the strict concavity of V implying strict quasi-concavity of the ratio. □

Theorem 7.3 (Budget separation)**.** *Let B^*_{CVIH} maximize CVIH. Let B^*_Q denote the quality-optimal budget, defined as follows: if $Q^*(B)$ attains a finite maximum (as occurs when quality functions are non-monotone with a peak), then $B^*_Q = \arg\max_B Q^*(B)$; if $Q^*(B)$ is strictly increasing for all B (as occurs for monotone quality functions with an asymptote at 1), then $B^*_Q = \infty$. In either case:*

$$B^*_{CVIH} < B^*_Q. \tag{17}$$

Proof. Case 1: $B^*_Q < \infty$ (non-monotone quality). At B^*_Q, we have $(Q^*)'(B^*_Q) = 0$. Since $V(B) = Q^*(B)$ is concave with $V'(B^*_Q) = 0$ and $V(B^*_Q) > 0$, the ratio $V(B)/B$ is strictly decreasing at B^*_Q (its derivative is $(V'B - V)/B^2 = -V/B^2 < 0$). Hence $B^*_{\text{CVIH}} < B^*_Q$.

Case 2: $B^*_Q = \infty$ (monotone quality with asymptote). The function $V(B)/B$ has a finite maximizer because V is bounded (by 1, since $Q^* \leq 1$) while $B \to \infty$, so $V(B)/B \to 0$. Since $V(B)/B$ starts positive and tends to zero, it must achieve its maximum at some finite $B^*_{\text{CVIH}} < \infty = B^*_Q$. □

Interpretation. Theorem 7.3 formalizes a crucial operational insight: the budget that maximizes quality is not the budget that maximizes efficiency. Spending more always (weakly) improves quality, but past B^*_{CVIH}, each additional dollar buys less verified output than the average dollar already spent. The CVIH-optimal budget represents the "sweet spot" where value per dollar is maximized.

7.2. Budget Sufficiency and Graceful Degradation

When the available budget B_{avail} falls below B^*_{CVIH}, the system must degrade gracefully.

Conjecture 7.1 (Graceful degradation)**.** *Under the TBAP optimal allocation, if the budget is reduced from B^* to αB^* (with $\alpha \in (0,1)$), the quality degrades sub-linearly in the log domain:*

$$\frac{\log Q^*(\alpha B^*)}{\log Q^*(B^*)} \leq \frac{1}{\alpha}, \tag{18}$$

that is, a fraction α of the budget retains more than a fraction α of the log-quality.

Remark. *This conjecture is motivated by the concavity of $\log Q^*(B)$ (proposition 3.4), which implies that the log-quality curve bends downward: earlier dollars contribute more than later dollars. We have verified this numerically for exponential saturation quality functions, where a 50% budget cut ($\alpha = 0.5$) retains 65–80% of the log-quality depending on the number of stages and saturation rate. A formal proof would require bounding the curvature of $\log Q^*$ from below, which depends on the specific quality function family. We leave this as an open problem.*

8. Caching Economics

8.1. Cache Hit Rate Model

Prompt caching is the single most impactful cost optimization available to LLM-based systems. When a prefix of the prompt matches a cached entry, the provider charges a reduced rate.

Definition 8.1 (Effective price with caching)**.** *The effective per-token price at stage k with caching is:*

$$p_k^{eff} = p_k \cdot (1 - \rho_k + \rho_k \cdot \delta_k)\,, \tag{19}$$

where $\rho_k \in [0,1]$ is the cache hit rate at stage k and $\delta_k \in (0,1)$ is the cache discount factor (the fraction of the full price charged for cached tokens).

For example, with $\rho_k = 0.92$ (92% cache hit rate) and $\delta_k = 0.10$ (90% discount on cached tokens), the effective price is $p_k^{\text{eff}} = p_k \cdot (1 - 0.92 + 0.92 \times 0.10) = p_k \cdot 0.172$, an 82.8% cost reduction.

8.2. Empirical Caching Data

Recent empirical data confirms the magnitude of caching benefits:

- **Claude Code (Anthropic):** Reports 90–95% prompt cache hit rates in typical agentic coding sessions, with an effective cost reduction of approximately 81% (Anthropic, 2026).
- **Cross-provider analysis:** Liang et al. (2026) document 41–80% cost reductions across providers using prompt-aware caching strategies.
- **Agentic plan caching:** Zhang et al. (2025) demonstrate a complementary approach, caching not just prompt prefixes but entire plan structures, achieving approximately 50% cost reduction and 27% latency reduction for recurring task patterns.

8.3. The Cache-First Principle

Proposition 8.1 (Cache-First principle)**.** *Under current LLM provider implementations (where cached tokens produce identical model computation to uncached tokens), caching reduces cost with zero quality impact. Formally, increasing ρ_k reduces p_k^{eff} without affecting $q_k(r_k)$, since the cached tokens deliver identical model behavior. All other cost levers (reducing r_k, switching to a cheaper model, compressing prompts) reduce q_k along with cost.*

Remark. *The Cache-First principle is an empirical property of current provider implementations, not a mathematical necessity. If a provider implemented approximate caching with lossy compression, cached tokens could produce different behavior, and the zero-quality-impact property would no longer hold. Under the current exact-caching paradigm, the principle follows directly: $q_k(r_k)$ is invariant to ρ_k, while $C(\mathrm{r}) = \sum_k p_k^{eff} r_k$ is strictly decreasing in ρ_k for $\delta_k < 1$.*

Corollary 8.1. *The Cache-First principle implies a strict ordering of cost optimization activities: maximize cache hit rates before pursuing any optimization that trades quality for cost.*

9. Design Principles

We now distill the formal results into seven actionable design principles for economics-aware harness architecture. Each principle is grounded in a theorem or empirical result from the preceding sections.

9.1. Principle 1: Equal Marginal Rate

Statement: Equalize the marginal quality-per-dollar across all pipeline stages.

Basis: Theorem 3.1 (Equal Marginal Rate theorem).

Implementation: Monitor $q_k'(r_k)/(q_k(r_k) \cdot p_k)$ for each stage. If this ratio is significantly higher for some stage j than for stage k, shift budget from k to j. In practice, this means: do not over-invest in code generation while starving the planning or testing stages.

9.2. Principle 2: Raise the Floor

Statement: Invest disproportionately in the weakest pipeline stage.

Basis: Theorem 3.3 (Weakest-Link theorem) and corollary 3.1.

Implementation: Identify the stage with the lowest $q_k(r_k)$ and prioritize budget increases there. A pipeline with qualities $(0.95, 0.92, 0.88, 0.60, 0.93, 0.91)$ has aggregate quality $Q = 0.382$. Improving the weakest stage (0.60) to 0.80 yields $Q = 0.509$, a 33% improvement. Improving the best stage (0.95) to 0.99 yields $Q = 0.398$, only a 4% improvement.

9.3. Principle 3: Cap the Context

Statement: Set a maximum useful token budget per stage, beyond which additional tokens may harm quality.

Basis: Empirical evidence from Gao et al. (2025) and Liu et al. (2024) showing that LLM performance degrades with excessive context length ("lost in the middle" and "context rot" effects; Chroma Research, 2025).

Implementation: For each stage, establish an empirical ceiling $r_k^{\max}$ beyond which quality is non-monotone. Enforce $r_k \leq r_k^{\max}$ as a hard constraint in the TBAP. This modifies axiom **(Q2)** for practical use: quality is monotone only up to $r_k^{\max}$.

9.4. Principle 4: Detect the Regime

Statement: Use different models for interactive and autonomous workloads.

Basis: The dual-regime cost function (section 5) and the regime-optimal model selection result.

Implementation: Classify each task as interactive or autonomous. For interactive tasks, select the cheapest model with response time below the flow-state threshold ($w_{\text{thresh}} \approx 120$–$180$ seconds). For autonomous tasks, select the model with the best cost-per-quality ratio regardless of latency.

9.5. Principle 5: Cache First

Statement: Maximize cache hit rates before pursuing any cost optimization that trades quality for cost.

Basis: Proposition 8.1 (Cache-First principle).

Implementation: Structure prompts with stable prefixes to maximize prompt cache hit rates. Implement plan-level caching for recurring task patterns (Zhang et al., 2025). Design harness data flow to preserve cacheable segments across iterations.

9.6. Principle 6: Route by Economics

Statement: Model selection should be driven by cost-effectiveness, not capability alone.

Basis: The JASP (definition 4.2) and the worked example in section 4.

Implementation: Maintain a cost-capability matrix for available models. For each stage, select the model that minimizes cost subject to the minimum capability threshold. This often means using cheaper models for stages where the frontier model provides little incremental quality (formatting, simple retrieval, boilerplate generation).

9.7. Principle 7: Budget for Jevons

Statement: Expect total token spend to increase when per-token prices fall; plan accordingly.

Basis: Theorem 6.1 and corollary 6.1 (Jevons paradox).

Implementation: When projecting token budgets, do not assume that falling prices will reduce costs. Model demand expansion at both the intensive margin (more tokens per task) and the extensive margin (new use cases becoming viable). Budget forecasts should include an elasticity multiplier: if prices fall by factor x, expect spend to change by factor $x^{1-\epsilon}$ (which increases for $\epsilon > 1$).

10. Empirical Calibration

The formal results in sections 3 to 7 require empirical parameterization. This section provides calibration data from April 2026.

Table 2 | Selected model pricing, April 2026 (per million tokens). Prices are approximate and subject to change. Cache discount is the fraction of full price charged for cached tokens.

Provider	Model	Input ($/Mtok)	Output ($/Mtok)	Cache Discount
Anthropic	Claude Opus 4	15.00	75.00	0.10
Anthropic	Claude Sonnet 4	3.00	15.00	0.10
Anthropic	Claude Haiku 3.5	0.80	4.00	0.10
OpenAI	GPT-4.1	2.00	8.00	0.50
OpenAI	GPT-4.1 mini	0.40	1.60	0.50
OpenAI	GPT-4.1 nano	0.10	0.40	0.50
OpenAI	o3	10.00	40.00	–
Google	Gemini 2.5 Pro	1.25	10.00	0.25
Google	Gemini 2.5 Flash	0.15	0.60	0.25

10.1. Quality Function Families

Two parametric families fit observed LLM quality-vs-token curves:

Exponential saturation.

$$q(r) = 1 - \exp(-\alpha r), \tag{20}$$

where $\alpha > 0$ controls the rate of quality improvement. This satisfies all five axioms **(Q1)–(Q5)**. Typical values: $\alpha \in [0.5 \times 10^{-4}, 5 \times 10^{-4}]$ when r is measured in tokens.

Hill function.

$$q(r) = \frac{r^n}{r^n + K_d^n}, \tag{21}$$

where $n > 0$ is the Hill coefficient and $K_d > 0$ is the half-saturation constant. For $n \leq 1$, this is concave. The Hill function better captures the empirical observation that very small token allocations produce near-zero quality (sharper threshold than exponential).

Both families are empirical approximations. The true quality-vs-token relationship depends on model architecture, task type, prompt structure, and many other factors. We recommend fitting parameters per-stage from logged pipeline data rather than assuming universal constants.

10.2. Token Pricing Data (April 2026)

Key observations from table 2:

- The price spread from cheapest to most expensive input pricing is approximately 150× (GPT-4.1 nano at $0.10/Mtok versus Claude Opus 4 at $15.00/Mtok).
- Cache discounts vary significantly: Anthropic offers 90% off for cached tokens, OpenAI 50%, Google 75%.
- The three-year trend (April 2023 to April 2026) shows approximately 300× reduction in effective cost at the frontier tier when caching is included.

10.3. Caching Parameters

Empirical caching parameters for agentic coding workflows:

- **Prompt cache hit rate:** 85–95% for iterative coding sessions with stable system prompts and tool definitions (Anthropic, 2026; Liang et al., 2026).
- **Plan cache hit rate:** 40–70% for recurring task patterns (e.g., "add a REST endpoint," "write unit tests for module X") (Zhang et al., 2025).
- **Effective cost reduction:** 60–85% when prompt caching and plan caching are combined.

10.4. Developer Time Data

For the queueing economics of section 5:

- **Fully loaded developer cost:** $90–$200/hr (including salary, benefits, overhead, tooling). The range reflects variation across markets, seniority levels, and organizational overhead factors.
- **Flow-state threshold:** 120–180 seconds. Responses faster than this allow the developer to maintain focus; slower responses trigger context-switching (Leroy, 2009; Mark et al., 2008).
- **Context-switching cost:** 15–30 minutes of reduced productivity per interruption (Mark et al., 2008).

10.5. The Calibration Protocol

We recommend the following protocol for calibrating the economics architecture:

1. **Measure:** Instrument the pipeline to log per-stage token consumption, latency, quality proxies (test pass rate, review acceptance rate), and cache hit rates.
2. **Fit:** Estimate quality function parameters (α, K_d, n) per stage from logged data using maximum likelihood or method of moments.
3. **Optimize:** Solve the JASP (definition 4.2) using fitted parameters and current pricing data.
4. **Verify:** Run A/B tests comparing the optimized allocation against the current allocation. Measure CVIH improvement.
5. **Repeat:** Re-calibrate monthly or when pricing changes by more than 20%.

11. Related Work

Model routing and cost optimization. Chen et al. (2023) introduced the idea of LLM cascades, where queries are routed to increasingly expensive models only when cheaper models fail, achieving up to 98% cost savings with minimal quality loss. Madaan et al. (2024) extended this to automatic mixing of language models based on query complexity, learning routing policies from preference data. Ong et al. (2024) formalized the routing problem and proposed learned preference-based routers that achieve cost-quality Pareto improvements. Our work differs in scope: these systems optimize per-query model selection, while we optimize per-stage allocation within a multi-stage pipeline, addressing the joint allocation-selection problem.

Prompt and plan caching. Liang et al. (2026) provided the first systematic study of prompt-aware caching strategies across LLM providers, documenting 41–80% cost reductions and identifying cache-friendly prompt design patterns. Zhang et al. (2025) introduced plan-level caching for agentic workflows, caching not just prompts but entire reasoning plans, achieving 50% cost and 27% latency

reductions. Our Cache-First principle (proposition 8.1) provides the formal justification for prioritizing these optimizations.

Context length and quality. Gao et al. (2025) demonstrated that increasing context length alone does not improve code agent performance and can actively hurt it, contradicting the assumption that more context is always better. Liu et al. (2024) identified the "lost in the middle" phenomenon where LLMs underutilize information placed in the middle of long contexts. Chroma Research (2025) documented "context rot," where agent performance degrades systematically with context accumulation over long sessions. These findings motivate our "Cap the Context" principle and the non-trivial shape of empirical quality functions.

Classical economics and optimization. The TBAP draws on portfolio theory (Markowitz, 1952) (optimal allocation across assets with different risk-return profiles), the Kelly criterion (Kelly, 1956) (optimal bet sizing under multiplicative returns), and the water-filling solution from information theory (Cover and Thomas, 2006). The queueing analysis builds on Kleinrock (1975, 1976) and the Erlang loss model (Erlang, 1917). The Jevons paradox formalization follows Jevons (1865) with modern treatment from Sorrell (2009) and Alcott (2005).

Value of information. Howard (1966) formalized the value of information in decision theory, and Blackwell (1953) established the comparison of experiments framework. Our CVIH metric can be viewed as an operational instantiation of the value-of-information concept, where each pipeline stage's token allocation represents an "experiment" whose output informs subsequent stages.

Submodular optimization. The multiplicative quality structure connects to submodular function maximization (Buchbinder et al., 2015; Feige et al., 2011; Krause and Golovin, 2014; Nemhauser et al., 1978), though our specific structure (product of concave functions subject to a linear constraint) is more tractable than the general submodular case. Sviridenko (2004) provides relevant approximation results for submodular maximization under knapsack constraints.

Developer productivity and AI tools. Peng et al. (2023) measured the productivity impact of GitHub Copilot, finding 55% faster task completion. METR (2025) studied early-2025 AI tool impact on developer productivity in realistic settings. These studies provide context for the developer time parameters in our queueing model but do not address the economics architecture of the tools themselves.

Behavioral economics. Kahneman and Tversky (1979) and Thaler (1999) established that economic agents exhibit systematic biases (loss aversion, mental accounting) that deviate from rational optimization. While our formal framework assumes rational optimization, we acknowledge in section 13 that behavioral factors (e.g., sunk cost bias leading to over-investment in failing pipelines, mental accounting of "AI budget" as separate from "infrastructure budget") may distort real-world adoption of these principles.

12. Contrarian Positions

We engage four common objections to economics-focused harness design, evaluating each on a 5-point strength scale.

12.1. "Quality Over Cost"

Claim: Always use the best available model; cost optimization compromises output quality.

Strength: 5/5 for safety-critical applications (medical, financial, security-sensitive code); 3/5 for general software engineering.

Evidence: The claim is strongest where errors have catastrophic consequences. For security-sensitive code, IBM and Ponemon Institute (2025) estimates the average cost of a data breach at $4.5M, dwarfing any token cost savings. However, for the median coding task (refactoring, feature implementation, test writing), the quality gap between frontier and mid-tier models is narrowing. SWE-bench data shows that mid-tier models achieve 70–85% of frontier pass rates at 10–20% of the cost (JetBrains, 2025).

Resolution: Tiered quality. Use the JASP (definition 4.2) with stage-specific capability thresholds: frontier models for security-critical stages, mid-tier for the rest. This is not "quality over cost" or "cost over quality" but rather "appropriate quality at appropriate cost."

12.2. "Falling Prices Make Optimization Unnecessary"

Claim: Token prices are falling so fast that cost optimization is a wasted effort; just wait six months.

Strength: 2/5.

Evidence: While prices have indeed fallen approximately 300× in three years, the Jevons paradox (theorem 6.1) with estimated $\epsilon \approx 1.19$ means total spend increases as prices fall. The bandwidth market provides the closest historical analogue: per-bit costs fell by over 1000× from 1995 to 2015, yet total telecom spending increased throughout, and traffic engineering became a specialized discipline.

Resolution: The "falling prices" argument confuses per-unit cost with total cost. Cost optimization addresses total cost, which is a function of both price and demand. With elastic demand ($\epsilon > 1$), falling prices make optimization more necessary, not less.

12.3. "Developer Time Dominates Token Cost"

Claim: Developer time is so expensive relative to token cost that optimizing token spend is not worth the engineering effort.

Strength: 4/5 for interactive workflows (correct in the interactive regime); 1/5 for autonomous workflows (incorrect in the autonomous regime).

Evidence: In the interactive regime, a developer earning $150/hr who waits 30 seconds for each response incurs $1.25 in waiting cost per interaction. At mid-tier pricing with caching ($0.50/Mtok effective), a 50K-token interaction costs $0.025 in token costs. Developer time dominates by 50×. However, in the autonomous regime (overnight batch runs, CI/CD pipelines, parallel task execution), developer waiting cost is zero. A 100-iteration autonomous run consuming 5M tokens costs $2.50–$75 in tokens with no developer time cost, and at organizational scale (100 such runs per day), the annual token budget reaches $100K–$2.7M.

Resolution: Regime detection (section 5). The claim is correct for interactive work, where optimizing for latency dominates optimizing for token cost. It is incorrect for autonomous work, where token cost is the only cost. A well-designed harness distinguishes the two regimes.

12.4. "Routing Complexity Outweighs Savings"

Claim: The engineering complexity of multi-model routing, caching optimization, and budget allocation is not worth the savings.

Strength: 3/5 for small-scale deployments (fewer than 10 developers); 1/5 for large-scale deployments (more than 100 developers).

Evidence: At small scale, the absolute savings from optimization may be $500–$2,000/month, which is less than the engineering cost of building and maintaining the optimization layer. At large scale, savings of $20K–$200K/month easily justify dedicated infrastructure. The FinOps survey (FinOps Foundation, 2023) found that organizations spending over $1M/year on cloud services save 20–30% from active cost optimization, well above the cost of the optimization effort.

Resolution: Tiered routing complexity. Small deployments should use simple rules (e.g., "frontier model for planning, economy model for everything else"). Medium deployments should implement the JASP with periodic recalibration. Large deployments should invest in real-time optimization with continuous monitoring. The framework in this paper supports all three tiers.

13. Limitations

Quality function estimation. The quality functions $q_k(\cdot)$ must be estimated empirically from pipeline execution data. The parametric families in section 10 (exponential saturation, Hill function) are approximations; real quality functions may have non-concave regions (especially at very large or very small token counts), discontinuities (from model-specific behavior), or task-dependent shapes. Misspecification of quality functions leads to suboptimal allocation.

Stage independence. The multiplicative quality model (eq. (1)) assumes that each q_k depends only on its own token allocation r_k, not on upstream outputs. In practice, a poor planning stage (stage 1) not only reduces q_1 but also degrades the effective quality function $q_k(\cdot)$ for all downstream stages. A more realistic model would write $q_k(r_k, q_{k-1})$, creating a sequential dependency chain. This correlated model is substantially harder to optimize (the log-transformed objective is no longer separable). We conjecture that the EMR condition (theorem 3.1) and the Weakest-Link result (theorem 3.3) are qualitatively robust to moderate correlations: the incentive to raise the floor on the weakest stage is amplified (not diminished) when weak stages propagate failures downstream. Formally verifying this robustness for specific correlation structures is an important direction for future work.

Static optimization. The TBAP and JASP solve a static optimization problem: given a task, allocate budget optimally before execution begins. In practice, the harness could dynamically reallocate budget during execution based on intermediate results (e.g., allocate more tokens to testing if initial tests fail). Dynamic reallocation is a richer but harder problem (stochastic dynamic programming) that we leave to future work.

Pricing volatility. The empirical calibration in section 10 reflects April 2026 pricing. Model prices change frequently (new model releases, competitive pricing moves, promotional discounts). The optimization must be reparameterized whenever prices change materially, which could be monthly or even weekly.

Behavioral factors. We assume rational optimization throughout. Real decision-makers exhibit biases documented by Kahneman and Tversky (1979) and Thaler (1999): sunk cost bias (continuing to invest in a failing pipeline), mental accounting (treating "AI budget" differently from equivalent spending categories), and loss aversion (preferring the certainty of a high-cost, high-quality model over the uncertainty of an optimized mixed-model pipeline). These behavioral factors may impede adoption of formally optimal strategies.

14. Conclusion

We have presented a formal economics architecture for AI coding agent harnesses, grounded in the structural asymmetry between multiplicative quality and additive cost. The Token Budget Allocation Problem yields an Equal Marginal Rate theorem and a Weakest-Link result that together prescribe equalization of quality-per-dollar across pipeline stages. The Model Tier Selection Problem, while NP-hard in general, is tractable by enumeration for the small instance sizes encountered in practice. Queueing analysis of the dual-regime cost function (interactive versus autonomous) reveals that optimal model selection is regime-dependent, with different models preferred in each regime. The Jevons paradox, confirmed empirically for token markets ($\epsilon \approx 1.19$), implies that cost optimization becomes more valuable as prices fall. The CVIH metric provides a principled efficiency measure whose optimum lies strictly below the quality optimum, formalizing the trade-off between quality and cost-effectiveness.

The seven design principles derived from these results (Equal Marginal Rate, Raise the Floor, Cap the Context, Detect the Regime, Cache First, Route by Economics, Budget for Jevons) provide actionable guidance for harness architects. The Cache-First principle is particularly significant: prompt and plan caching offer 60–85% cost reductions with zero quality penalty, making them the unambiguous first priority for any cost optimization effort.

As token prices continue to fall and agentic workflows continue to expand, the economics architecture of AI coding harnesses will become as critical as their capability architecture. The framework presented here provides the formal foundation for that economic layer.

Appendices

A. Proofs

A.1. Proof of the Equal Marginal Rate Theorem (theorem 3.1)

Proof. Consider the log-transformed TBAP:

$$\underset{\mathrm{r}>0}{\text{maximize}} \sum_{k=1}^{K} \log q_k(r_k) \quad \text{subject to} \quad \sum_{k=1}^{K} p_k r_k \le B.$$

The Lagrangian is:

$$\mathcal{L}(\mathrm{r}, \lambda) = \sum_{k=1}^{K} \log q_k(r_k) - \lambda \left(\sum_{k=1}^{K} p_k r_k - B \right).$$

Slater's constraint qualification holds: the point $\mathrm{r} = (B/(2Kp_k))_k$ is strictly feasible (it uses only half the budget). Therefore, by the KKT theorem (Bertsekas, 1999; Boyd and Vandenberghe, 2004),

there exists $\lambda^* \geq 0$ such that:

$$\left.\frac{\partial \mathcal{L}}{\partial r_k}\right|_{\mathrm{r}^*} = \frac{q_k'(r_k^*)}{q_k(r_k^*)} - \lambda^* p_k = 0 \quad \forall k.$$

Rearranging:

$$\frac{q_k'(r_k^*)}{q_k(r_k^*) \cdot p_k} = \lambda^* \quad \forall k.$$

The complementary slackness condition $\lambda^*(B - \sum_k p_k r_k^*) = 0$ implies that either $\lambda^* = 0$ (in which case each q_k is individually maximized, which contradicts the budget constraint being tight for any finite B when quality functions are strictly increasing) or the budget constraint is tight: $\sum_k p_k r_k^* = B$. In the non-degenerate case, $\lambda^* > 0$ and the budget is fully spent.

The second-order sufficiency condition holds because $\log q_k$ is concave (proposition 3.2), ensuring r^* is a global maximum. □

A.2. Proof of the Weakest-Link Theorem (theorem 3.3)

Proof. We prove the result in two steps.

Step 1: AM-GM bound. By the inequality of arithmetic and geometric means (Hardy et al., 1952), for non-negative reals $a_1, \ldots, a_K$:

$$\left(\prod_{k=1}^{K} a_k\right)^{1/K} \leq \frac{1}{K}\sum_{k=1}^{K} a_k,$$

with equality if and only if $a_1 = a_2 = \cdots = a_K$. Setting $a_k = q_k(r_k)$ gives eq. (7).

Step 2: Connection to budget-constrained optimization. Consider two allocations r and r' with the same total cost $C(\mathrm{r}) = C(\mathrm{r}') = B$, but suppose r yields stage qualities $(q_1, \ldots, q_K)$ with $q_j < q_k$ for some stages j, k, while r' achieves more equalized qualities. The AM-GM inequality implies:

$$\prod_{k=1}^{K} q_k(r_k') \leq \left(\frac{1}{K}\sum_{k=1}^{K} q_k(r_k')\right)^K,$$

with equality when all qualities are equal. Since the arithmetic mean is a linear function of the individual qualities, and quality functions are concave, reallocating budget from high-quality stages to low-quality stages ("raising the floor") increases the product while keeping the budget constant. The optimal allocation equalizes the marginal rates (theorem 3.1), which, in the symmetric case, yields equal allocation and hence equal qualities. □

A.3. Proof of Pareto Frontier Log-Concavity (proposition 3.4)

Proof. Define $g(B) = \log Q^*(B) = \max_{\mathrm{r}}\{\sum_k \log q_k(r_k) : \sum_k p_k r_k \leq B, \mathrm{r} > 0\}$.

For any $B_1, B_2 > 0$ and $\alpha \in [0, 1]$, let r_1^* and r_2^* be optimal for budgets B_1 and B_2 respectively. The convex combination $\mathrm{r}_\alpha = \alpha \mathrm{r}_1^* + (1-\alpha)\mathrm{r}_2^*$ is feasible for budget $\alpha B_1 + (1-\alpha)B_2$ (by linearity of the cost

function). By concavity of each $\log q_k$:

$$\begin{aligned} g(\alpha B_1 + (1-\alpha)B_2) &\geq \sum_k \log q_k(\alpha r^*_{1,k} + (1-\alpha) r^*_{2,k}) \\ &\geq \sum_k \left[\alpha \log q_k(r^*_{1,k}) + (1-\alpha)\log q_k(r^*_{2,k})\right] \\ &= \alpha g(B_1) + (1-\alpha) g(B_2). \end{aligned}$$

Therefore $g(B) = \log Q^*(B)$ is concave, i.e., Q^* is log-concave.

For property (3), by the envelope theorem, $g'(B) = \lambda^*(B)$, where $\lambda^*(B)$ is the optimal Lagrange multiplier at budget B. Since $g = \log Q^*$, we have $\frac{d}{dB} \log Q^*(B) = \lambda^*(B)$. □

A.4. Proof of CVIH Quasi-Concavity (theorem 7.2)

Proof. Let $h(B) = V(B)/B$ where V is concave, $V(0) = 0$, and $V'(0) > 0$. We show h is quasi-concave with a unique maximizer.

Quasi-concavity. The upper level sets $\{B > 0 : h(B) \geq c\} = \{B > 0 : V(B) \geq cB\}$ are the intersection of the epigraph of V (a concave function) with the half-space above the line cB. This intersection is convex (it is an interval $[B_l(c), B_u(c)]$ for each c, since $V(B) - cB$ is concave). Therefore h is quasi-concave.

Existence and uniqueness of maximizer. Since $V(0) = 0$:

$$\lim_{B \to 0^+} h(B) = \lim_{B \to 0^+} \frac{V(B)}{B} = V'(0^+) > 0,$$

by L'Hôpital's rule (or the definition of the derivative). Since V is bounded (quality is bounded by 1 and iteration rate is bounded), $h(B) \to 0$ as $B \to \infty$. The function h is continuous on $(0, \infty)$, positive near zero, and vanishes at infinity, so it attains a maximum at some $B^* > 0$.

For uniqueness, note that $h'(B) = [V'(B)B - V(B)]/B^2$. The numerator $\phi(B) = V'(B)B - V(B)$ satisfies $\phi'(B) = V''(B)B < 0$ for all $B > 0$ (since V is strictly concave, $V'' < 0$). Therefore ϕ is strictly decreasing, so $\phi(B) = 0$ has at most one solution, implying $h'(B) = 0$ has a unique solution B^*. □

B. Empirical Data

B.1. Complete Pricing Tables

Table 3 | Anthropic model pricing, April 2026 (per million tokens).

Model	Input	Output	Cache Write	Cache Read
Claude Opus 4	$15.00	$75.00	$18.75	$1.50
Claude Sonnet 4	$3.00	$15.00	$3.75	$0.30
Claude Haiku 3.5	$0.80	$4.00	$1.00	$0.08

Table 4 | OpenAI model pricing, April 2026 (per million tokens).

Model	Input	Output	Cached Input	Cache Discount
o3	$10.00	$40.00	$5.00	50%
o4-mini	$1.10	$4.40	$0.55	50%
GPT-4.1	$2.00	$8.00	$1.00	50%
GPT-4.1 mini	$0.40	$1.60	$0.20	50%
GPT-4.1 nano	$0.10	$0.40	$0.05	50%

Table 5 | Google model pricing, April 2026 (per million tokens, for prompts up to 200K tokens).

Model	Input	Output	Cache Discount
Gemini 2.5 Pro	$1.25	$10.00	75%
Gemini 2.5 Flash	$0.15	$0.60	75%
Gemini 2.0 Flash	$0.10	$0.40	75%

B.2. Caching Mechanism Comparison

Table 6 | Caching mechanisms across providers and approaches.

Approach	Scope	Cost Reduction	Latency Reduction	Quality Impact	Source
Prompt prefix caching	Token-level	60–85%	50–80%	None	Anthropic (2026)
Prompt-aware caching	Token-level	41–80%	30–60%	None	Liang et al. (2026)
Agentic plan caching	Plan-level	~50%	~27%	Minimal	Zhang et al. (2025)
Semantic deduplication	Embedding-level	20–40%	10–20%	Low	Various

B.3. Model Routing Results Summary

Table 7 | Summary of model routing approaches and their reported savings.

System	Routing Strategy	Cost Savings	Quality Retention	Source
FrugalGPT	LLM cascade (cheap to expensive)	up to 98%	95–100%	Chen et al. (2023)
AutoMix	Learned complexity routing	50–70%	90–98%	Madaan et al. (2024)
RouteLLM	Preference-based routing	40–75%	95–99%	Ong et al. (2024)

References

B. Alcott. Jevons' paradox. *Ecological Economics*, 54(1):9–21, 2005.

Anthropic. Agentic coding best practices. Technical report, Anthropic, 2026. URL https://docs.anthropic.com/en/docs/build-with-claude/agentic-coding.

R. E. Barlow and F. Proschan. *Statistical Theory of Reliability and Life Testing: Probability Models*. Holt, Rinehart and Winston, 1975.

D. P. Bertsekas. *Nonlinear Programming*. Athena Scientific, 2nd edition, 1999.

D. Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2): 265–272, 1953.

S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

N. Buchbinder, M. Feldman, J. S. Naor, and R. Schwartz. Submodular maximization with cardinality constraints. In *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1433–1452, 2015.

C. Chekuri and S. Khanna. A polynomial time approximation scheme for the multiple knapsack problem. *SIAM Journal on Computing*, 35(3):713–728, 2005.

L. Chen, M. Zaharia, and J. Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

Chroma Research. Context rot: How long contexts degrade agent performance. Technical report, Chroma, 2025.

T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 2nd edition, 2006.

A. K. Erlang. Solution of some problems in the theory of probabilities of significance in automatic telephone exchanges. *Elektroteknikeren*, 13:5–13, 1917.

U. Feige, V. S. Mirrokni, and J. Vondrák. Maximizing non-monotone submodular functions. *SIAM Journal on Computing*, 40(4):1133–1153, 2011.

FinOps Foundation. State of FinOps survey 2023. Technical report, FinOps Foundation, 2023.

Y. Gao et al. Context length alone hurts: Rethinking long-context utilization in LLM-based code agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

G. H. Hardy, J. E. Littlewood, and G. Pólya. *Inequalities*. Cambridge University Press, 2nd edition, 1952.

R. A. Howard. Information value theory. *IEEE Transactions on Systems Science and Cybernetics*, 2(1): 22–26, 1966.

IBM and Ponemon Institute. Cost of a data breach report 2025. Technical report, IBM Security, 2025.

JetBrains. SWE-bench agent complexity analysis. Technical report, JetBrains Research, 2025.

W. S. Jevons. *The Coal Question: An Inquiry Concerning the Progress of the Nation, and the Probable Exhaustion of Our Coal-Mines*. Macmillan and Co., London, 1865.

D. Kahneman and A. Tversky. Prospect theory: An analysis of decision under risk. *Econometrica*, 47 (2):263–291, 1979.

J. L. Kelly, Jr. A new interpretation of information rate. *The Bell System Technical Journal*, 35(4): 917–926, 1956.

L. Kleinrock. *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.

L. Kleinrock. *Queueing Systems, Volume 2: Computer Applications*. Wiley-Interscience, 1976.

A. Krause and D. Golovin. Submodularity and its applications in optimized information gathering. In *Tractability: Practical Approaches to Hard Problems*, pages 71–104. Cambridge University Press, 2014.

S. Leroy. Why is it so hard to do my work? the challenge of attention residue when switching between work tasks. *Organizational Behavior and Human Decision Processes*, 109(2):168–181, 2009.

T. Liang et al. Don't break the cache: Prompt-aware caching strategies for large language model serving. *arXiv preprint arXiv:2601.06007*, 2026.

N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

A. Madaan, P. Aggarwal, P. Anand, S. Potdar, S. Mishra, P. Zhou, A. Gupta, D. Rajalingham, S. Muckatira, Mausam, and A. Grover. AutoMix: Automatically mixing language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

G. Mark, D. Gudith, and U. Klocke. The cost of interrupted work: More speed and stress. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08)*, pages 107–110. ACM, 2008.

H. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.

A. Mas-Colell, M. D. Whinston, and J. R. Green. *Microeconomic Theory*. Oxford University Press, 1995.

METR. Measuring the impact of early-2025 AI on developer productivity. Technical report, METR, 2025.

G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978.

I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*, 2023.

R. T. Rockafellar. *Convex Analysis*. Princeton University Press, 1970.

S. Sorrell. Jevons' paradox revisited: The evidence for backfire from improved energy efficiency. *Energy Policy*, 37(4):1456–1469, 2009.

M. Sviridenko. A note on maximizing a submodular set function subject to a knapsack constraint. *Operations Research Letters*, 32(1):41–43, 2004.

R. H. Thaler. Mental accounting matters. *Journal of Behavioral Decision Making*, 12(3):183–206, 1999.

J. A. Van Mieghem. Dynamic scheduling with convex delay costs: The generalized $c\mu$ rule. *The Annals of Applied Probability*, 5(3):809–833, 1995.

Y. Zhang et al. Agentic plan caching for software engineering tasks. *arXiv preprint arXiv:2506.14852*, 2025.